

SKALA Python 데이터 분석 실습 결과 보고서

실습 4·5 · 종합실습 2·3 · 추가과제

4기_광주_3반_채수아

GitHub: github.com/sua918/skala-assignment

목차

1부. 데이터 분석 실습

실습 4 · Pandas 2.x 데이터 정제

실습 5 · Pandas·Polars·DuckDB 성능 비교

2부. 종합 실습

종합실습 2 · EDA·통계·ML 파이프라인

종합실습 3 · 분석 자동화·HTML 리포트

3부. 추가과제

스키마 버전 호환·DLQ 재처리·드리프트 원인 추적

4부. 결과 및 자기 평가

1. 실습 4 — Pandas 2.x 데이터 정제

판매 데이터 5,000건의 타입 불일치, 결측값과 이상치를 정제하고 카테고리 집계, 지역별 피벗과 담당자 마스터 병합을 수행했습니다.

1.1 정제 및 집계 구조

- 날짜·수량·단가·할인을 명시적으로 변환하고 범주형 컬럼을 category로 관리했습니다.
- 단가는 카테고리별 중앙값, 수량은 전체 중앙값, 할인은 0으로 대체했습니다.
- 잘못된 날짜와 결측 카테고리, 그룹 중앙값을 구할 수 없는 경우에는 전역 대체와 오류 처리를 적용했습니다.
- IQR 상한·하한을 계산해 행을 삭제하지 않는 원저라이징으로 극단값을 조정했습니다.
- groupby.agg, pivot_table, left merge 후 병합 전후 행 수와 미연결 담당자를 검증했습니다.

1.2 실행 방법

```
python Python_data_0721_Day2/practice4/solution.py
```

```
=====
실습 4 | Pandas 2.x 데이터 정제
=====
행 수          : 5,000
결측값         : 516 → 0
단가 최댓값    : 299,608 → 299,608
수량 최댓값    : 1,995 → 15.5
총 판매출     : 4,086,433,016원
병합 행 수     : 5,000 → 5,000
담당자 미지정  : 0건

정제 후 데이터 타입
order_id       str
order_date     datetime64[us]
region         category
category       category
quantity       float64
unit_price     float64
discount       float64
amount         float64

카테고리별 집계
      orders  avg_price  total_quantity  total_sales
category
Food        1049  150797.78           5843.0  8.386387e+08
Beauty      1012  156526.85           5576.5  8.254606e+08
Fashion      967  155762.03           5453.0  8.126457e+08
Home         977  150478.74           5661.5  8.059198e+08
Electronics  995  150542.30           5717.5  8.037681e+08

지역 x 카테고리 피벗
category      Beauty  Electronics  Fashion  Food  Home
region
Busan      146943834.0  151189082.0  158875148.0  141523900.0  149401872.0
Daegu      136325750.0  155807805.0  165496819.0  163426671.0  154978967.0
Gwangju    152904323.0  172159853.0  161182056.0  160460528.0  146502087.0
Incheon    175629378.0  143914687.0  147460739.0  153660439.0  148712465.0
Seoul      167888224.0  135152955.0  124080997.0  164849311.0  150409802.0
Unknown    45769108.0   45543743.0   55549958.0   54717876.0   55914641.0

정제 결과 저장          : practice4\output
```

실습 4 — 정제 전후와 집계 실행 결과

1.3 실행 결과

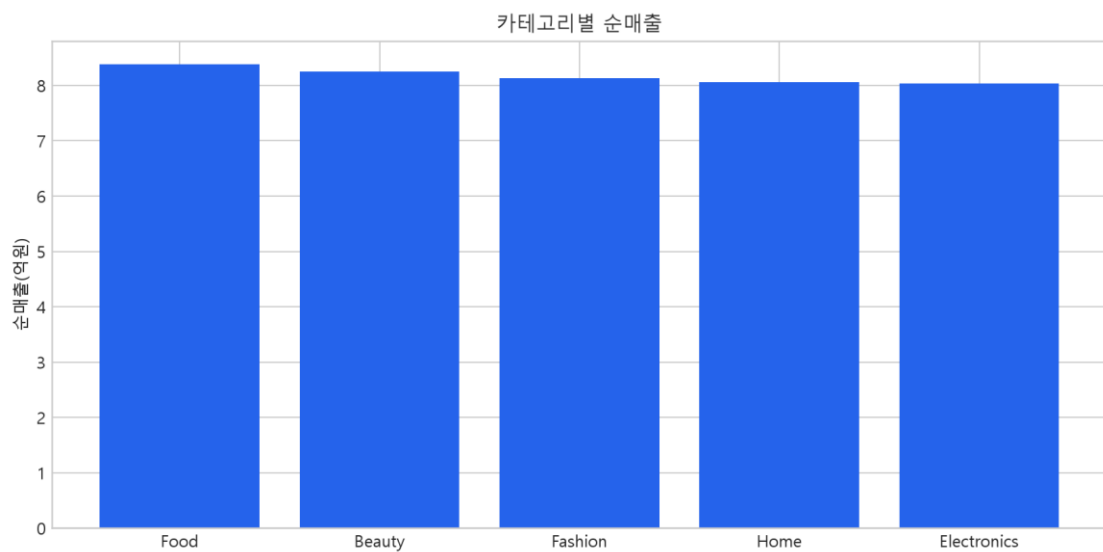
처리 행 수: 5,000건

결측값: 516건 → 0건

수량 최댓값: 1,995 → 15.5

총 순매출: 4,086,433,016원

병합 검증: 5,000건 → 5,000건, 미연결 0건



실습 4 — 카테고리별 순매출 비교

1.4 생성 파일

- output/sales_clean.csv — 정제된 판매 데이터 5,000건
- output/category_summary.csv — 카테고리별 주문·단가·수량·매출 집계
- output/region_category_pivot.csv — 지역×카테고리 매출 피벗
- output/merge_sample.csv — 담당자 마스터 병합 결과
- output/metrics.json — 정제 전후 지표와 병합 무결성 검증

1.5 확인한 점

- 타입 정규화 후 결측 처리를 수행해야 변환 실패로 생긴 결측까지 포함할 수 있습니다.
- 원저라이징은 행의 다른 정보를 보존하면서 극단값이 집계에 미치는 왜곡을 줄입니다.
- 병합 후 행 수와 미연결 값을 함께 검사해야 조용한 데이터 손실을 방지할 수 있습니다.

2. 실습 5 — Pandas·Polars·DuckDB 성능 비교

100만 건 이벤트 데이터에 동일한 집계를 Pandas, Polars Lazy API와 DuckDB SQL로 실행하고 결과가 일치한 뒤에만 성능을 비교했습니다.

2.1 비교 구조

- amount가 0보다 큰 이벤트를 event_type별로 묶어 건수·평균 금액·총금액을 계산했습니다.
- Polars는 scan_csv 기반 Lazy 실행, DuckDB는 CSV 직접 SQL 조회를 사용했습니다.
- 정렬·자료형·컬럼을 통일한 뒤 assert_frame_equal로 세 엔진 결과의 동일성을 검증했습니다.
- 엔진별 워밍업 후 실행 순서를 회전하며 3회 측정한 중앙값을 비교했습니다.
- 소형 고정 CSV 입력에서도 세 실제 엔진이 같은 결과를 반환하는 테스트를 추가했습니다.

2.2 실행 방법

```
python Python_data_0721_Day2/practice5/solution.py
```

```
=====
실습 5 | Pandas · Polars · DuckDB 성능 비교
=====
비교 질의      : 제공 데이터에는 status/value 컬럼이 없으므로 amount > 0인 이벤트만 골라 event_type별 건수·평균 금액·총금액을 집계한다.
세 엔진 결과 일치 검증 : PASS

집계 결과
event_type event_count avg_amount total_amount
purchase   60062      251706.06  1.511797e+10
refund     19844      250129.10  4.963562e+09

벤치마크 중앙값 (3회)
engine median_seconds relative_to_fastest
Polars Lazy      0.0526      1.00
DuckDB           0.2088      3.97
Pandas           0.9644     18.32

비교 결과 저장      : practice5\output
```

실습 5 — 세 엔진 결과 일치 및 벤치마크

2.3 실행 결과

결과 일치: Pandas = Polars Lazy = DuckDB (PASS)

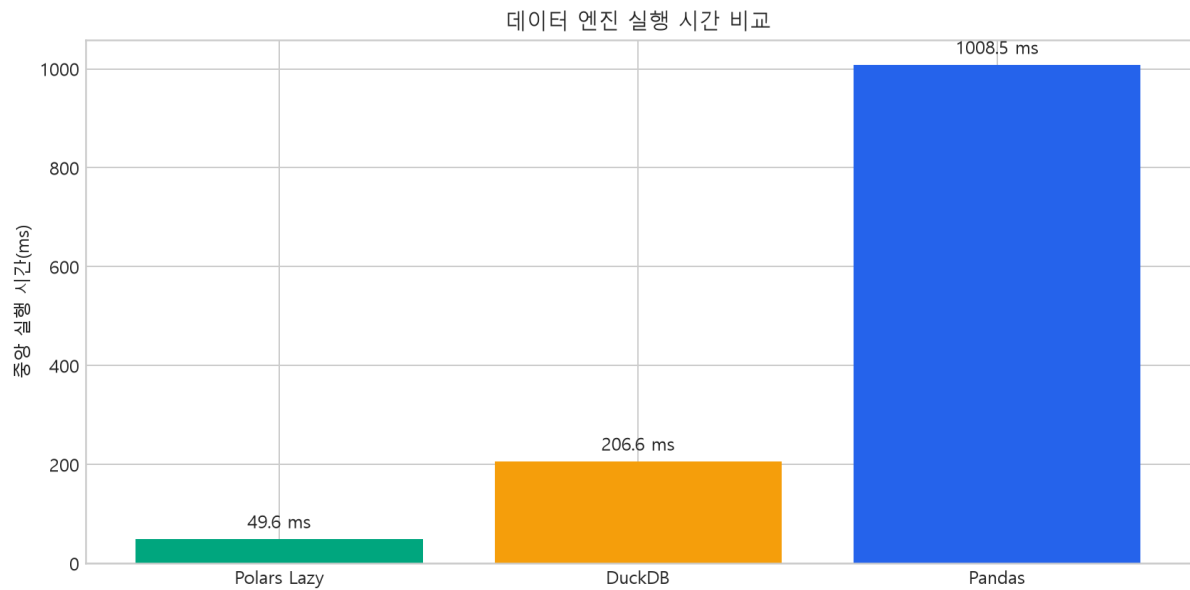
Polars Lazy: 약 0.0508초

DuckDB: 약 0.2022초

Pandas: 약 1.0574초

가장 빠른 엔진: Polars Lazy

집계 대상: purchase 60,062건, refund 19,844건



실습 5 — 엔진별 중앙 실행 시간 비교

2.4 생성 파일

- output/aggregation_result.csv — 세 엔진 공통 집계 결과
- output/benchmark.csv — 중앙 실행 시간과 최단 시간 대비 배수
- output/polars_lazy_plan.txt — 최적화된 Lazy 실행 계획
- output/metrics.json — 질의 정의, 결과 일치 여부와 엔진별 실행 시간

2.5 확인한 점

- 성능을 비교하기 전에 엔진별 집계 결과의 동일성을 먼저 증명해야 합니다.
- Polars Lazy는 필요한 행과 컬럼만 읽도록 실행 계획을 최적화합니다.
- 성능 수치는 환경의 영향을 받으므로 절대값보다 동일 조건과 측정 절차가 중요합니다.

3. 종합실습 2 — EDA·통계·ML 파이프라인

고객 이탈 데이터 7,000건을 탐색하고 통계적 연관성을 확인한 뒤, 전처리와 예측 모델을 하나의 재현 가능한 Pipeline으로 학습·저장했습니다.

3.1 분석 및 모델 구조

- 수치형·범주형 분포, 결측률과 이탈률을 EDA 표와 시각화로 확인했습니다.
- 월 요금은 독립표본 t-검정, 계약 유형은 카이제곱 검정으로 이탈 여부와의 연관성을 검정했습니다.
- train/test 분리 후 ColumnTransformer에서 결측 처리·표준화·One-Hot 인코딩을 수행했습니다.
- RandomForest를 Pipeline에 포함해 전처리 누수를 방지하고 전체 객체를 joblib로 저장했습니다.
- 결측 churn 타깃은 입력 단계에서 차단하고, 저장 모델을 다시 불러 ROC-AUC 재현성을 검증했습니다.

3.2 실행 방법

```
python Python_data_0721_Day2/Total2/analysis.py
```

```
=====
종합실습 2 | EDA + 통계 + ML 파이프라인
=====
분석 고객 수      : 7,000명
이탈률           : 24.07%
t-검정 p-value    : 1.226e-20
카이제곱 p-value  : 1.325e-70
ROC-AUC          : 0.6475
이탈 Precision    : 0.3433
이탈 Recall       : 0.4777
이탈 F1           : 0.3995
모델 재로딩 검증  : PASS
해석             : 월 요금과 계약 유형은 이탈 여부와 유의한 연관성을 보이지만 인과를 의미하지 않는다.
산출물           : Total2\output
=====
```

종합실습 2 — 통계 검정과 모델 성능

3.3 실행 결과

분석 고객 수: 7,000명

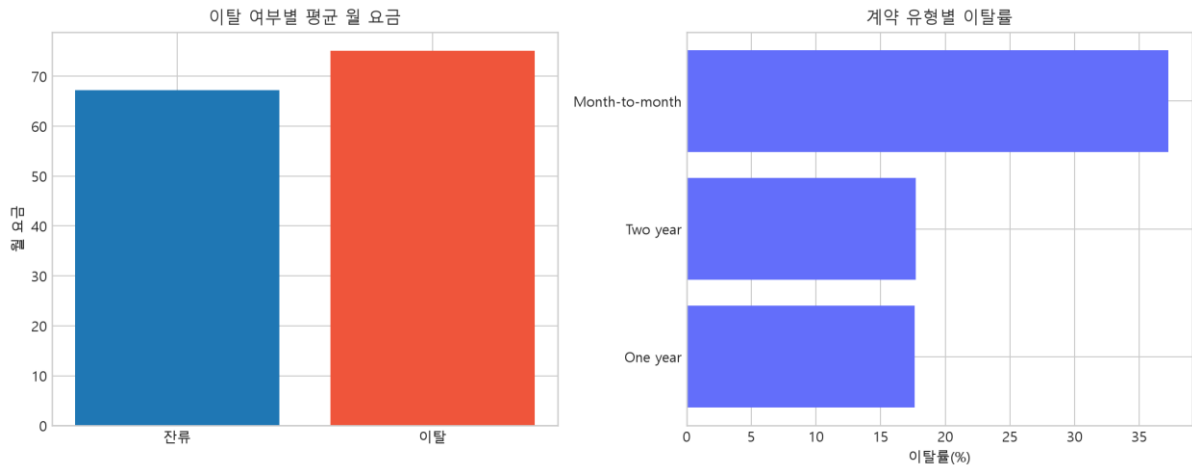
이탈률: 24.07%

t-검정 p-value: 1.226e-20

카이제곱 p-value: 1.325e-70

ROC-AUC: 0.6475

저장 모델 재검증: PASS



종합실습 2 — 이탈 여부별 월 요금과 계약 유형별 이탈률

3.4 생성 파일

- output/eda_summary.csv — 변수별 탐색 지표
- output/churn_overview.png — 핵심 EDA 시각화
- output/churn_eda_report.html — 탐색 결과 HTML
- output/churn_pipeline.joblib — 전처리와 모델이 결합된 Pipeline
- output/metrics.json — 검정 통계량, 분류 지표와 모델 재검증 결과

3.5 확인한 점

- 유의한 p-value는 변수 간 연관성을 보여주지만 인과관계를 의미하지 않습니다.
- 전처리와 모델을 함께 저장해야 새 데이터에서도 동일한 변환 순서를 재현할 수 있습니다.
- 저장 직후 재로딩 평가를 수행하면 배포 산출물과 기록 지표의 불일치를 조기에 발견할 수 있습니다.

4. 종합실습 3 — 분석 자동화·HTML 리포트

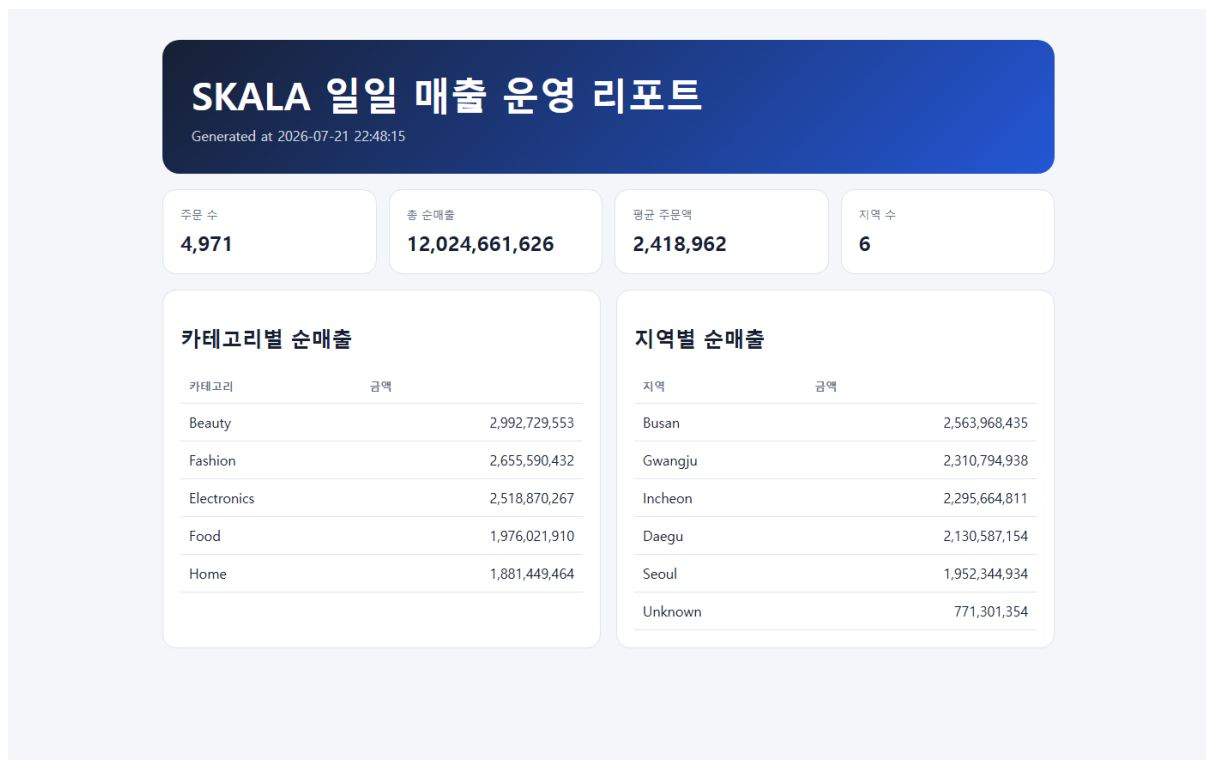
설정·정제·집계·렌더링·실행 주기를 분리하고, 실행할 때마다 이전 결과를 보존하는 타임스탬프 기반 매출 HTML 리포트를 생성했습니다.

4.1 자동화 구조

- frozen dataclass 설정과 순수 집계 함수, Jinja2 템플릿, 실행 조율 모듈로 역할을 분리했습니다.
- 필수 컬럼·빈 데이터·숫자 변환 결과를 검증하고, 할인율을 0~1 범위로 제한했습니다.
- 평균 주문액은 행 평균이 아니라 order_id별 주문 합계의 평균으로 계산했습니다.
- 같은 초에 재실행해도 순번을 붙여 파일 충돌 없이 이전 리포트를 보존합니다.
- 스케줄 실행 중 오류는 로그로 격리하고 다음 실행 주기를 계속하도록 구성했습니다.

4.2 실행 방법

```
python Python_data_0721_Day2/Total3/report.py
```



4.3 실행 결과

주문 수: 4,971건

총 순매출: 12,024,661,626원

평균 주문액: 2,418,962원 (주문별 합계 기준)

지역 수: 6개

파일 보존: 타임스탬프와 충돌 방지 순번 적용

4.4 생성 파일

- output/sales_report_YYYYMMDD_HHMMSS.html — 실행 시각별 매출 리포트
- templates/report.html — 반응형 리포트 레이아웃과 KPI 카드
- config.py — 입력·출력·템플릿 경로와 상위 항목 수 설정
- run_scheduler.py — 예외 격리와 반복 실행을 담당하는 스케줄러

4.5 확인한 점

- 설정·분석·표현을 분리하면 각 단계를 독립적으로 테스트하고 교체할 수 있습니다.
- 업무 KPI는 계산 단위를 먼저 정의해야 하며 평균 주문액은 주문 단위로 집계해야 합니다.
- 자동화 작업은 한 번의 오류가 전체 스케줄을 종료하지 않도록 실패 경계를 설계해야 합니다.

5. 추가과제 — 스키마 호환·DLQ 재처리·드리프트 원인 추적

실습 4·5와 종합실습 2·3의 흐름을 확장해, 버전이 다른 고객 이벤트를 표준 스키마로 수용하고 실패 이벤트를 재처리한 뒤 데이터 변화·모델 영향·원인 후보를 한 화면에서 추적하도록 구현했습니다.

5.1 기존 과제에서 발전시킨 구조

- 실습 4의 타입 정규화와 그룹·전역 대치를 기준·신규 기간에 동일하게 적용했습니다.
- 실습 5의 비교 방식을 확장해 Pandas·Polars·DuckDB 집계 결과를 자동 교차 검증했습니다.
- 종합실습 2의 이탈 Pipeline으로 신규 데이터의 ROC-AUC·Recall·예측 위험도 변화를 추적했습니다.
- 종합실습 3의 설정·분석·렌더링 분리 구조로 CSV·JSON·모델·HTML을 일괄 생성했습니다.
- 입력 단계에 스키마 버전 변환, DLQ 상태 이력, 재처리와 event_id 기반 멍등성을 추가했습니다.

5.2 실행 방법

```
python Python_data_0721_Day2/Advanced/drift_monitor/main.py --scenario mixed_shift
```

5.3 스키마 호환과 DLQ 재처리 결과

신규 이벤트 수신: 3,501건

스키마별 수용: v1 1,750건 / v2 1,749건

초기 DLQ: 4건

재처리: 복구 3건 / 미해결 보존 1건

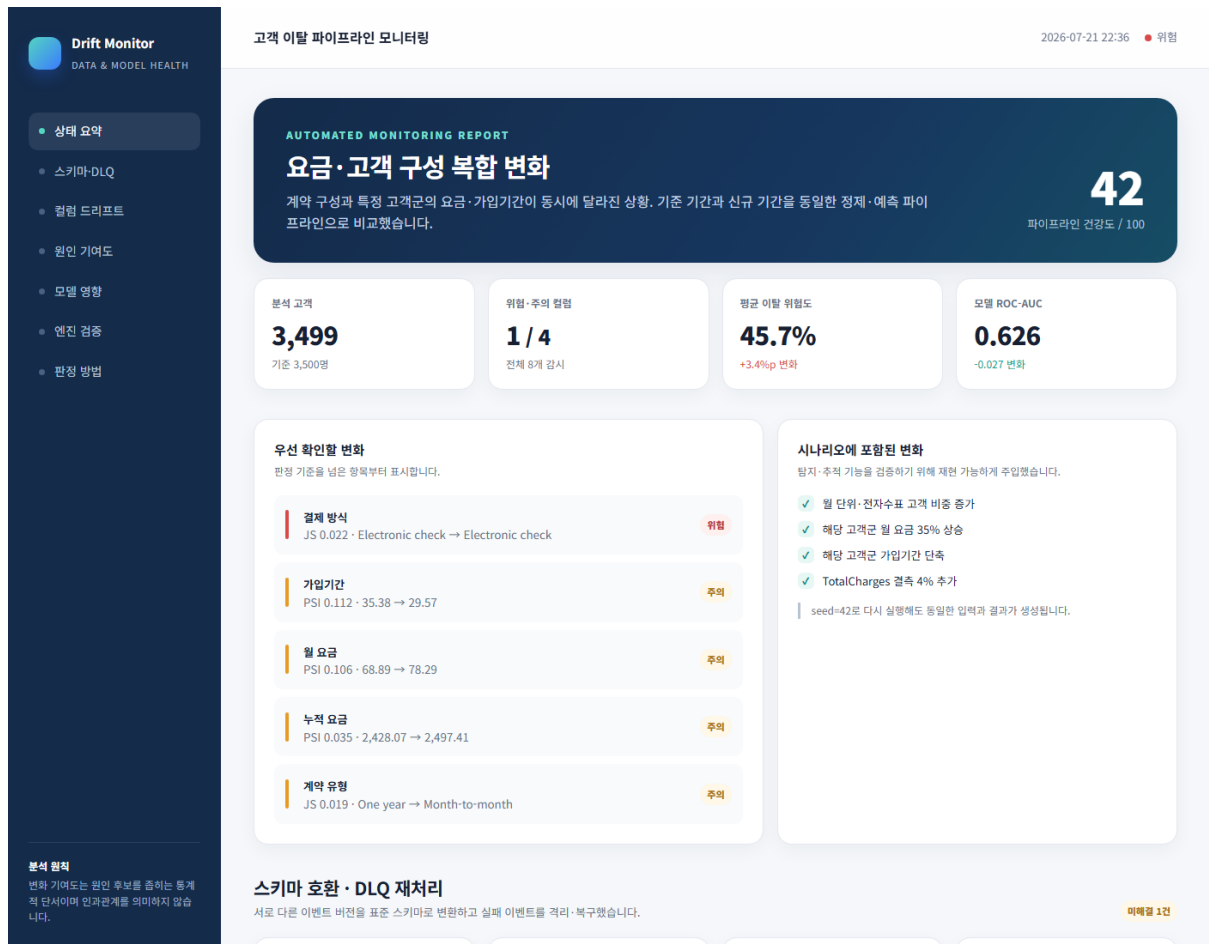
멍등성: 중복 1건 차단

최종 분석 데이터: 3,499건

- 평면 v1과 중첩 v2 이벤트를 customer_id 중심의 표준 분석 컬럼으로 변환했습니다.
- DLQ에는 실패 코드, 원본 이벤트, 재시도 횟수와 pending→recovered/failed 상태 이력을 보

존했습니다.

- 안전하게 추론할 수 있는 필드만 보정하고 고객 식별이 불가능한 이벤트는 failed 상태로 유지했습니다.



추가과제 — 스키마·DLQ·드리프트 통합 HTML 대시보드

5.4 드리프트와 모델 영향

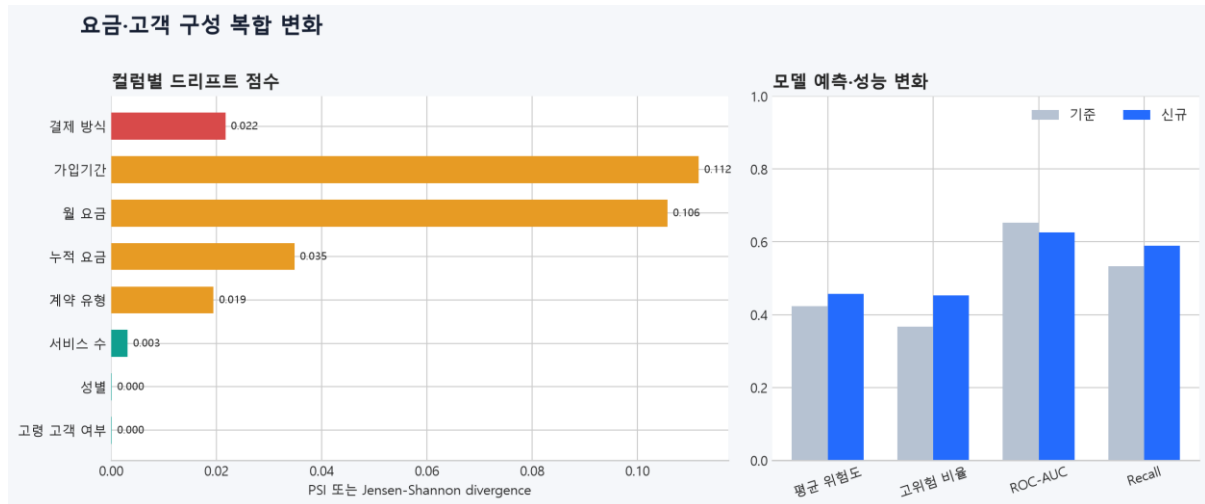
기준 / 신규 고객: 3,500명 / 3,499명

위험 / 주의 컬럼: 1개 / 4개

평균 이탈 위험도: 42.31% → 45.70%

ROC-AUC: 0.6531 → 0.6262

엔진 집계 결과: Pandas = Polars = DuckDB (PASS)



추가과제 — 컬럼 드리프트와 모델 지표 변화

5.5 드리프트 원인 자동 추적

- 변화가 큰 수치형 컬럼은 PSI·KS 통계량, 범주형 컬럼은 JS divergence와 신규 범주를 계산했습니다.
- 대상 변화량을 그룹 내부 변화와 고객 구성비 변화로 분해해 다음 확인 대상의 우선순위를 계산했습니다.
- 가입기간 감소의 1순위 후보는 payment_method=Electronic check이며 전역 정규화 기여도는 15%입니다.
- 소수 집단을 제외하고 모든 후보를 하나의 분모로 정규화해 기여도 합이 100%를 넘지 않게 했습니다.
- 기여도는 관측된 분포 변화의 설명 순위이며 정책 효과나 인과관계를 의미하지 않습니다.

```
=====
Advanced | 스키마·DLQ·드리프트 통합 모니터
=====
시나리오      : 요금·고객 구성 복합 변화
기준 / 신규 고객 : 3,500 / 3,499명
DLQ 최초 / 복구 / 미해결 : 4 / 3 / 1건
위험 / 주의 컬럼 : 1 / 4개
평균 이탈 위험도 : 42.31% → 45.70%
ROC-AUC       : 0.6531 → 0.6262
엔진 결과 검증 : PASS
HTML 리포트   : Advanced\drift_monitor\output\drift_monitor_report.html
```

추가과제 — 전체 실행 결과와 엔진 교차 검증

5.6 생성 파일

- output/schema_dlq_summary.json — 버전별 수용량과 DLQ 재처리 요약
- output/dlq_records.json — 실패 원본, 상태 이력과 재시도 결과
- output/drift_metrics.csv — 컬럼별 점수·상태·변화량
- output/root_causes.json — 대상 변수별 원인 후보와 기여도
- output/monitor_summary.json — 수집·품질·드리프트·모델·벤치마크 통합 결과
- output/drift_monitor_report.html — 키보드 탐색이 가능한 통합 대시보드
- output/churn_drift_pipeline.joblib — 기준 기간에서 학습한 이탈 Pipeline

6. 결과 및 자기 평가

6.1 검증 결과

실습 4: 결측 516→0, 병합 5,000→5,000, 미연결 0건 (PASS)

실습 5: 세 엔진 결과 일치, 중앙 실행 시간 비교 (PASS)

종합실습 2: 통계 검정 유의, ROC-AUC 0.6475, 저장 모델 재검증 (PASS)

종합실습 3: 주문 단위 평균·파일 충돌 방지·스케줄 오류 격리 (PASS)

추가과제: v1/v2 호환, DLQ 3/4건 복구, 중복 차단, 드리프트 원인 추적 (PASS)

자동 검사: 관련 범위 pytest 33개 전체 통과, Ruff 검사 통과

6.2 잘된 점

- 정제·집계·모델·자동화 결과를 콘솔 출력뿐 아니라 CSV·JSON·joblib·HTML로 재사용할 수 있게 구성했습니다.
- 성능 비교 전에 결과 일치를 검증하고, 모델 저장 후 재로딩 평가까지 수행해 재현성을 높였습니다.
- 추가과제에서 입력 버전 차이와 처리 실패를 흡수하고, 데이터 변화·모델 영향·우선 확인 고객 집단을 한 흐름으로 연결했습니다.

6.3 보완할 점

- ROC-AUC 개선을 위해 교차검증·하이퍼파라미터 탐색과 파생변수 설계가 필요합니다.
- 운영 환경에서는 실제 시간순 배치로 기준 구간을 관리하고 합성 시나리오를 대체해야 합니다.
- 원인 기여도는 통계적 설명이므로 인과 판단에는 별도의 실험과 업무 검토가 필요합니다.

6.4 종합 평가

실습 4·5의 정제·엔진 검증과 종합실습 2·3의 모델·자동화 구조를 운영 흐름으로 확장했습니다. 버전이 다른 입력과 실패 이벤트를 처리하고 데이터 변화·모델 영향·원인 후보·재처리 상태를 하나의 HTML 리포트에서 확인할 수 있도록 완성했습니다.